Track 6: Post-Quantum Cryptography

Lattices, Learning With Errors, and why noise protects secrets

Qiskit Fall Fest Hackathon · Oct 9, 2026 · IISER Bhopal (EECS Club and Physics Club)

Read this first. You have one week, a laptop, and AI assistants. The ten problem statements are suggestions, not a menu you must order from. You may pick one, extend one, combine several, or invent something more ambitious. Original, advanced and cross-track projects are welcome and will be judged on the same criteria.

What we reward. A project that works, that you can explain line by line, and that is honestly compared against a classical alternative. Using AI to write code is allowed and encouraged; being unable to explain or verify that code is not.

How to read the problem statements

Every problem has the same parts: a short background, the problem itself, the core tasks (what a complete submission does), stretch goals (what makes it excellent), and a hint on tools. Treat the core tasks as the minimum, not the ceiling.

Simulators first, hardware as a bonus. Everything here can be done on a simulator (Qiskit Aer or the statevector tools). Running a small version on real IBM hardware, with a clear comparison against the simulator, earns extra credit in any track.

This track contains: **Problem 10**.

Problem 10: Lattice-based cryptography from scratch

Background

Large quantum computers running Shor's algorithm would break the public-key systems used on the internet today, such as RSA and elliptic curve cryptography. Post-quantum cryptography replaces them with problems that are believed to be hard even for quantum computers. The leading family is based on lattices, and the problem called Learning With Errors (LWE): recover a secret from noisy linear equations. The new NIST standard for key exchange (ML-KEM, derived from Kyber) is a structured, efficient version of this idea.

Problem

Implement a small lattice-based encryption scheme, test its correctness, and explore where its security comes from.

Core tasks (the minimum for a complete submission)

1. Implement a toy LWE-based public-key encryption scheme (key generation, encryption of a bit or short message, decryption). Choose small parameters and write them down.
2. Measure the decryption failure rate as you vary the noise, and explain the trade-off between noise (security) and correctness.
3. Demonstrate an attack on deliberately tiny parameters, for example by lattice reduction (LLL) or by brute force, and show how the attack becomes infeasible as the dimension grows. Plot the cost against the dimension.
4. Explain, in plain words and with one figure, why adding noise turns an easy linear-algebra problem into a hard one.

Stretch goals (what makes it excellent)

- Move to the structured version: implement polynomial arithmetic with the number theoretic transform and build a simplified Kyber-style key exchange.
- Contrast with the threat: run Shor's algorithm in Qiskit to factor a small number such as 15 or 21, and relate the resource needs of breaking real keys to what hardware exists.
- Compare key and ciphertext sizes with RSA and elliptic curve schemes.
- Examine one implementation weakness, such as a timing leak in your own code, and fix it.

Hints on tools

This track is mostly classical code, so the quantum link is the story you tell: why Shor breaks the old systems and why no known quantum algorithm breaks lattice problems. State clearly that your toy scheme is for learning only and must never be used to protect real data.

Go beyond: combine tracks or invent your own

The most impressive entries will not stop at one box. None of these is required, and you are free to propose something entirely different.

Ideas that involve Track 6

Idea	Tracks combined	What makes it interesting
Quantum-safe key exchange pipeline	Lattice cryptography and QKD	Compare a lattice-based key exchange with QKD in security assumptions, cost and failure modes.

Open invitation. If you have your own idea, such as a quantum game, an algorithm tutor, a benchmark of a quantum algorithm from our workshop (Grover, phase estimation, the QFT), or a tool that makes any of the above easier for the next student, propose it. Write three sentences on the problem, the method, and how you will know it worked, and check with the organisers early in the week.

How to scope a combined project. Get the smallest version of each part working separately first, then connect them. A small, honest, working combination beats an ambitious one that does not run.

What every team submits

Whichever problem you choose (or invent), your submission has the same five parts:

- A code repository or notebook that runs from a clean start, with versions of the packages listed.
- A write-up PDF (two to four pages) covering the problem, method, results and an honest limits section.
- At least one figure that shows the main result.
- A short AI-use note: which tools you used, one thing the AI got wrong, and how you caught it.
- A five-minute demo followed by questions; every team member must be able to take questions.

Bonus. Bonus points are available for a hardware run on a real IBM backend, with a clear comparison against the simulator.

Full submission rules are in *docs/guidelines_and_submission_rules.pdf*.

How you are judged

The same scoring applies to every problem and to original projects alike.

Criterion	Points	What judges look for
Correctness and verification	30	Results checked against an exact or classical reference, tests, sensible error bars
Understanding and insight	25	Clear explanation of why the method works or fails, shown in the write-up and in the live questions
Honest comparison and limits	20	A fair classical baseline, stated assumptions, no unsupported claims of quantum advantage
Ambition and originality	15	Stretch goals, combined tracks, or a new idea beyond the suggested statements
Clarity of demo and code	10	Readable, reproducible code and a clear five-minute story
Total	**100**	Bonus points are available on top for a real IBM hardware run (see Bonus).

On using AI. AI assistants are welcome at every step. The AI-use note and the live questions exist because the person submitting is responsible for the result: you should be able to explain each circuit, formula and plot, and show how you verified it.